This is a copy of a chat between Claude and Lawrence. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. Report

JDK vs JRE vs JVM — differences & setup explain with example

JVM vs JRE vs JDK

These three are nested components — each builds on the previous one.

JVM — Java Virtual Machine

The **engine** that actually *runs* Java bytecode. It's an abstract machine that:

- Reads `.class` files (compiled bytecode)
- Translates them to native machine code at runtime (via JIT compiler)
- Manages memory (heap, stack, garbage collection)

Key point: JVM is platform-specific (Windows JVM ≠ Linux JVM), but the bytecode it runs is platform-independent — this is the "write once, run anywhere" promise.

JRE — Java Runtime Environment

JVM + standard libraries. Everything needed to *run* a Java program, but not compile one.

Contains:

- JVM
- Core class libraries (`java.lang`, `java.util`, `java.io`, etc.)
- Runtime support files

Use it if you just want to **run** `.jar` or `.class` files.

JDK — Java Development Kit

JRE + development tools. Everything needed to *write and compile* Java programs.

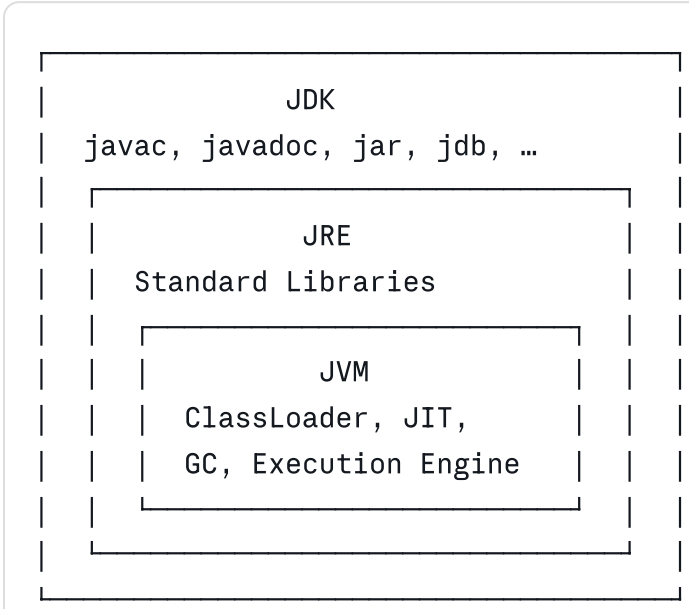
Contains:

- JRE (which includes JVM)

- `javac` — the compiler
- `javadoc`, `jdb` (debugger), `jar`, `jshell`, and more
- Header files for native code

Use it if you want to **build** Java applications.

Visual relationship



End-to-end example

1. You write source code → `Hello.java`

java

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

2. JDK's `javac` compiles it → produces `Hello.class` (bytecode)

bash

```bash
javac Hello.java      # needs JDK
```

3. JVM (via `java`) runs the bytecode

bash

```bash
java Hello            # needs JRE/JVM
# Output: Hello, World!
```

The `.class` file contains bytecode — not machine code. The JVM reads it and compiles it to native instructions at runtime (that's the JIT compiler doing its job).

Quick decision guide

Goal	You need
Run a `.jar` / `.class` file	JRE (or JDK)
Write & compile Java code	JDK
Deploy a Java app on a server	JRE (historically) or JDK
Use an IDE (IntelliJ, Eclipse)	JDK

> **Note:** Since Java 11, Oracle no longer ships a standalone JRE — the JDK is the standard install for everything. Tools like `jlink` let you bundle a minimal custom runtime for deployment.

Setup (quick)

```bash
# macOS (Homebrew)
brew install openjdk@21

# Ubuntu/Debian
sudo apt install openjdk-21-jdk

# Verify
java -version    # shows JVM version
javac -version   # confirms JDK tools are present
```

The most widely used distribution is **OpenJDK** (free, open-source). Others include Amazon Corretto, Eclipse Temurin, and GraalVM.

How Java code compiles and runs (bytecode, classloader)

How Java Code Compiles and Runs

The journey from `.java` source to running program has two distinct phases: **compile time** (done once) and **runtime** (done every execution).

Phase 1 — Compile Time (javac)

```
Hello.java  ──javac──▶  Hello.class
(source)                 (bytecode)
```

`javac` does several things:

- **Lexing & Parsing** — tokenizes source, builds an AST (Abstract Syntax Tree)
- **Semantic analysis** — type checking, resolving symbols
- **Bytecode generation** — emits platform-neutral `.class` file

The `.class` file is **not machine code** — it's an intermediate instruction set designed for the JVM, not your CPU.

bash

```
javac Hello.java       # produces Hello.class
javap -c Hello.class   # disassemble — peek at bytecode
```

Sample `javap` output:

```
public static void main(java.lang.String[]);
  Code:
     0: getstatic     #7   // Field System.out
     3: ldc           #13  // String "Hello, World!"
     5: invokevirtual #15  // Method println
     8: return
```

These are **JVM opcodes** — instructions like `getstatic`, `ldc`, `invokevirtual`. The JVM understands these regardless of OS.

Phase 2 — Runtime (JVM)

Once you run `java Hello`, the JVM takes over in three big steps:

Step 1 — ClassLoader Subsystem

The ClassLoader **finds, loads, links, and initializes** `.class` files on demand (lazy — only when first needed).

Three built-in ClassLoaders (Java 9+):

ClassLoader	Loads
Bootstrap	Core JDK classes (`java.lang.*`, `java.util.*`)
Platform	Extension/platform modules (`javax.*`, etc.)
Application	Your classes + classpath jars

They form a **parent-delegation chain**: before loading a class, each loader asks its parent first. This prevents you from accidentally overriding `java.lang.String`.

```
Your class request
       |
       ▼
Application ClassLoader
  → asks Platform ClassLoader
      → asks Bootstrap ClassLoader
          → found? return it
          → not found? try Platform
              → not found? try Application
                  → not found? ClassNotFoundException
```

Three sub-phases inside loading:

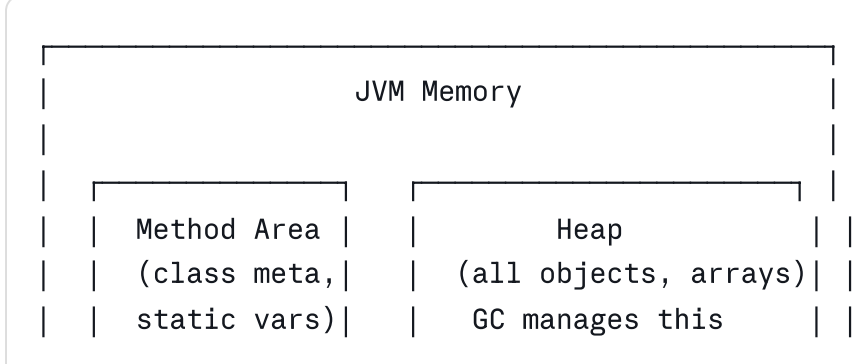
```
Load → Link → Initialize
```

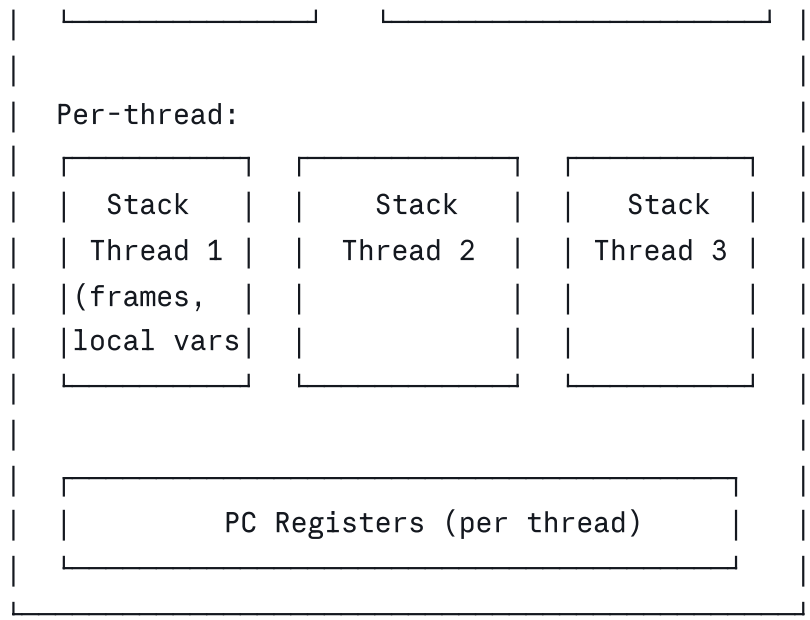
- **Load** — read the `.class` bytes into memory
- **Link**
  - *Verify* — validate bytecode is well-formed & safe
  - *Prepare* — allocate memory for static fields (set to defaults)
  - *Resolve* — replace symbolic references with direct memory references
- **Initialize** — run static initializers, assign static field values

---

Step 2 — Runtime Data Areas (Memory)

The JVM carves memory into distinct regions:



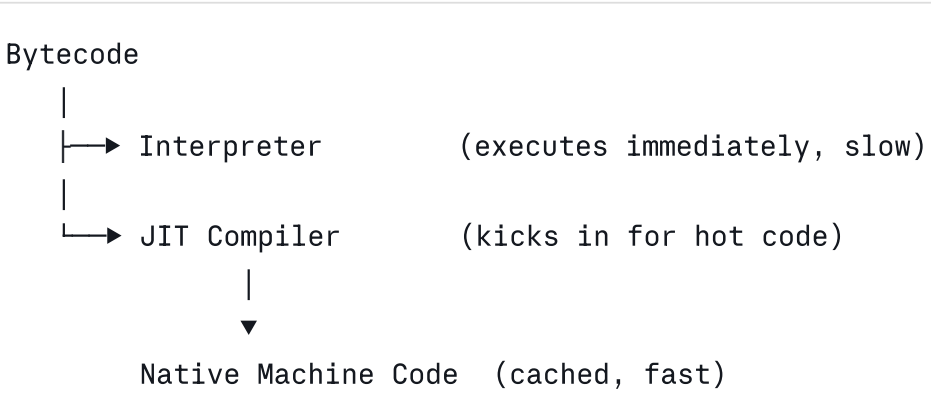


Area	What lives here
Heap	All `new` objects, arrays — shared across threads
Method Area	Class metadata, static fields, constant pool
Stack (per thread)	Stack frames — local variables, operand stack
PC Register	Points to current bytecode instruction for each thread
Native Stack	For native (JNI) method calls

Each **method call** pushes a new **stack frame**; return pops it.

Step 3 — Execution Engine

This is what actually *executes* the bytecode:



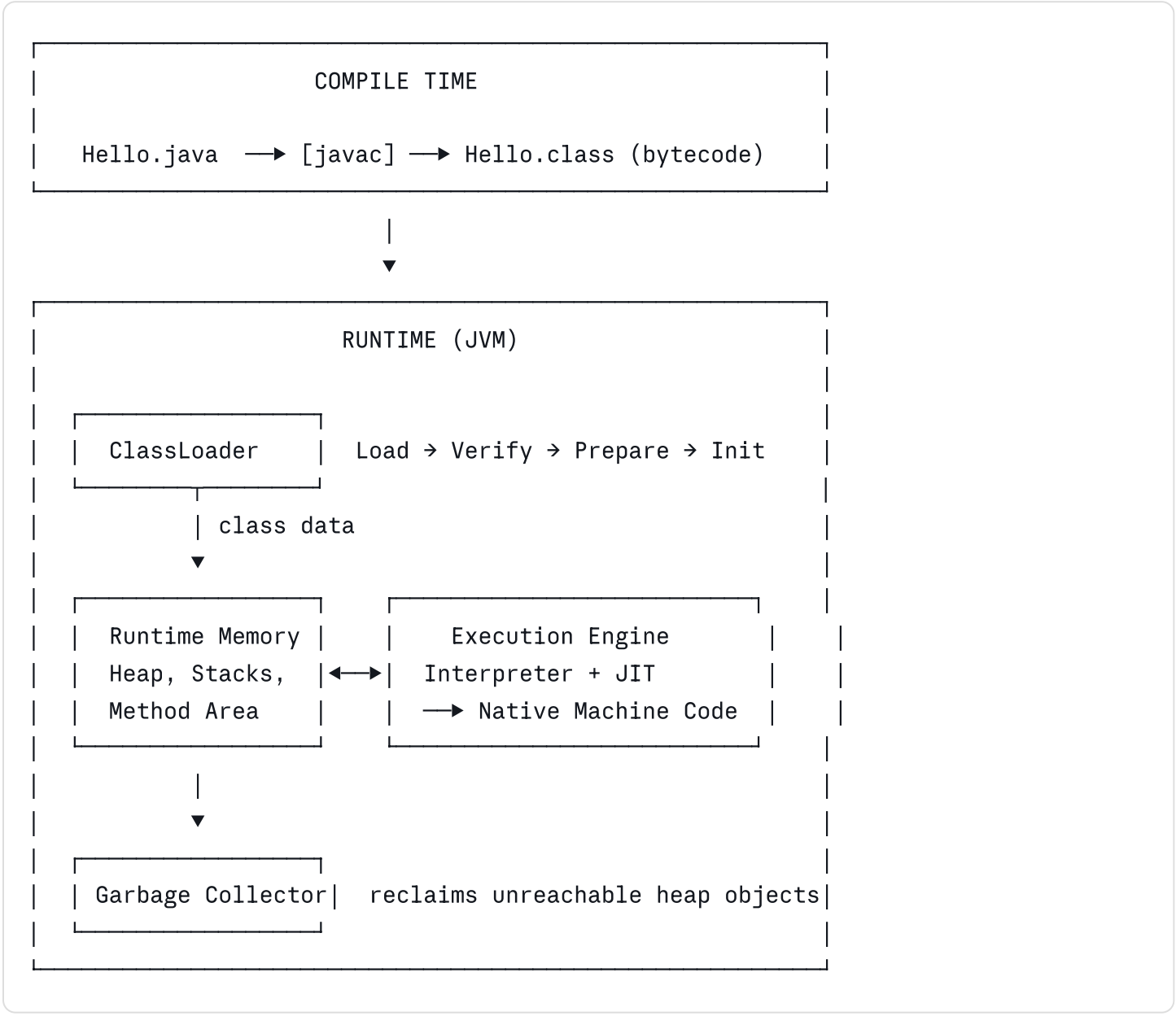
Interpreter runs bytecode line-by-line — fast to start, slow over time.

JIT (Just-In-Time) Compiler monitors which methods run frequently ("hot spots") and compiles them directly to native machine code. This is why Java gets *faster the longer it runs*.

The JIT applies aggressive optimizations:

- **Inlining** — replace method calls with the method body
- **Loop unrolling** — reduce loop overhead
- **Escape analysis** — allocate objects on stack instead of heap when possible
- **Dead code elimination**

Full picture — one diagram



Key takeaways

- `.java` → `.class` happens once via `javac`; the bytecode is OS-independent
- **ClassLoader** loads classes lazily using parent-delegation (security + isolation)
- **Heap** holds objects; **Stack** holds method frames — they serve different purposes
- **JIT** is why Java isn't slow — hot paths become native code at runtime
- **GC** runs in the background reclaiming heap memory; you never call `free()`

Class vs Object explain with example

Class vs Object in Java

Think of it this way:

- A **Class** is the **blueprint** — defined once, describes structure & behavior
- An **Object** is the **instance** — created from the blueprint, lives in memory

---

Class — the blueprint

java

```java
public class Car {

    // Fields — state (each object gets its own copy)
    String brand;
    String color;
    int speed;

    // Constructor — called when object is created
    public Car(String brand, String color) {
        this.brand = brand;
        this.color = color;
        this.speed = 0;
    }

    // Methods — behaviour
    public void accelerate(int amount) {
        speed += amount;
    }

    public void brake(int amount) {
        speed = Math.max(0, speed - amount);
    }

    public void status() {
        System.out.println(brand + " | " + color + " | " + speed + " km/h");
    }
}
```

The class **exists only in code** — no memory is allocated for objects yet.

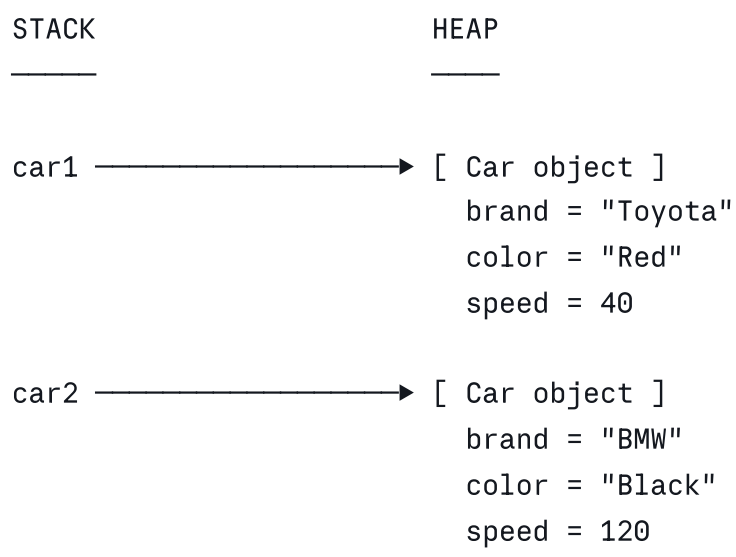
Object — the instance

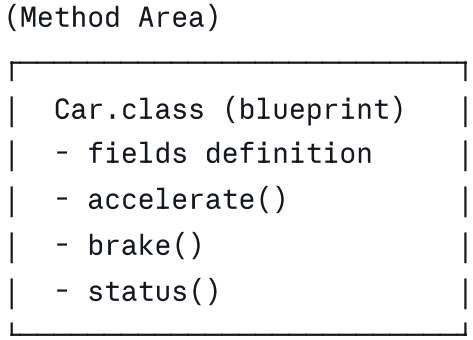
java

```java
public class Main {
    public static void main(String[] args) {

        // Create two objects from the same class
        Car car1 = new Car("Toyota", "Red");
        Car car2 = new Car("BMW", "Black");

        // Each object has its own independent state
        car1.accelerate(60);
        car2.accelerate(120);
        car1.brake(20);

        car1.status();   // Toyota | Red   | 40 km/h
        car2.status();   // BMW    | Black | 120 km/h
    }
}
```

`car1` and `car2` are two **separate objects** in heap memory — same blueprint, independent state.

What happens in memory





```
        ▲           ▲
      car1        car2
  (both share the same method code)
```

- **Variables** (`car1`, `car2`) live on the **stack** — they hold a *reference* (memory address), not the object itself
- **Objects** live on the **heap** — GC manages their lifecycle
- **Method code** lives in the **Method Area** — shared by all instances, not duplicated

---

Class members vs Instance members

java

```java
public class Car {

    // instance field — each object has its OWN copy
    String brand;

    // static field — ONE copy shared by ALL objects
    static int totalCars = 0;

    public Car(String brand) {
        this.brand = brand;
        totalCars++;          // shared counter increments for every new car
    }

    // instance method — works on THIS object's data
    public void show() {
        System.out.println("Car: " + brand);
    }

    // static method — belongs to class, no object needed
    public static void showTotal() {
        System.out.println("Total cars: " + totalCars);
    }
}
```

java

```java
Car c1 = new Car("Tesla");
Car c2 = new Car("Honda");
Car c3 = new Car("Ford");

c1.show();              // Car: Tesla
Car.showTotal();        // Total cars: 3
```

	Instance member	Static member
Belongs to	Each object	The class itself
Memory	Heap (per object)	Method Area (once)
Access via	`objectName.field`	`ClassName.field`
Use case	Per-object state/behaviour	Shared state, utility methods

---

`this` keyword

`this` refers to the **current object** — used to distinguish instance fields from parameters with the same name:

```java
public Car(String brand, String color) {
    this.brand = brand;   // this.brand = instance field
                          //      brand = constructor parameter
    this.color = color;
}
```

Key takeaways

	Class	Object
What	Blueprint / template	Instance in memory
When created	At compile time (`.class` file)	At runtime (`new` keyword)
Where	Method Area	Heap
How many	Once per program	As many as needed
Analogy	Cookie cutter	The actual cookie

> Every object **is-a** instance of its class, but a class **is not** an object — it's the definition. You can create thousands of `Car` objects from one `Car` class, each with its own state, all sharing the same method code.

Constructors (default & parameterized)
Constructor overloading

`this` keyword
Why do we need constructors?

Constructors in Java

A **constructor** is a special method that runs automatically when an object is created with `new`. Its job: **initialize the object's state.**

```java
Car c = new Car("Toyota");
//              ──────────
//              this triggers the constructor
```

Rules:

- Same name as the class
- No return type (not even `void`)
- Called once — at object creation time

Why do we need constructors?

Without a constructor, you'd have to manually set every field after creation — error-prone and easy to forget:

```java
// WITHOUT constructor — dangerous
Car c = new Car();
c.brand = "Toyota";   // what if you forget this?
c.speed = 0;          // object is in invalid state until all fields set
c.fuelLevel = 100;

// WITH constructor — safe, guaranteed initialization
Car c = new Car("Toyota", 100);
// object is fully valid the moment it's created
```

Constructors **enforce object validity** — an object either gets created correctly or not at all.

Default Constructor

A constructor with **no parameters.** If you write no constructor at all, Java silently inserts one for you.

```java
```

```java
public class Car {
    String brand;
    int speed;
    int fuelLevel;

    // Default constructor — no parameters
    public Car() {
        brand    = "Unknown";
        speed    = 0;
        fuelLevel = 100;
        System.out.println("Car created with defaults");
    }
}
```

java

```java
Car c = new Car();
// Output: Car created with defaults
// c.brand = "Unknown", c.speed = 0, c.fuelLevel = 100
```

> **Java's implicit constructor:** If you define *no* constructor, Java auto-generates `public Car() {}` — fields get default values (`null`, `0`, `false`). The moment you write *any* constructor yourself, Java stops generating one.

Parameterized Constructor

A constructor that **accepts arguments** so you can set fields at creation time:

java

```java
public class Car {
    String brand;
    String color;
    int speed;
    int fuelLevel;

    // Parameterized constructor
    public Car(String brand, String color, int fuelLevel) {
        this.brand     = brand;
        this.color     = color;
        this.fuelLevel = fuelLevel;
        this.speed     = 0;         // sensible default for speed
    }
}
```

java

```
Car c1 = new Car("Toyota", "Red",   100);
Car c2 = new Car("BMW",    "Black",  80);
// both objects are fully initialized immediately
```

Constructor Overloading

Just like method overloading — **multiple constructors, different parameter lists**. Java picks the right one based on the arguments you pass.

java

```
public class Car {
    String brand;
    String color;
    int speed;
    int fuelLevel;

    // Constructor 1 — no args, full defaults
    public Car() {
        this.brand     = "Unknown";
        this.color     = "White";
        this.fuelLevel = 100;
        this.speed     = 0;
    }

    // Constructor 2 — brand only
    public Car(String brand) {
        this.brand     = brand;
        this.color     = "White";   // default color
        this.fuelLevel = 100;
        this.speed     = 0;
    }

    // Constructor 3 — brand + color
    public Car(String brand, String color) {
        this.brand     = brand;
        this.color     = color;
        this.fuelLevel = 100;
        this.speed     = 0;
    }

    // Constructor 4 — full control
    public Car(String brand, String color, int fuelLevel) {
        this.brand     = brand;
        this.color     = color;
        this.fuelLevel = fuelLevel;
        this.speed     = 0;
```

```
    }

    public void status() {
        System.out.println(brand + " | " + color + " | fuel: " + fuelLevel + "% | "
    }
}
```

java

```
Car c1 = new Car();
Car c2 = new Car("Toyota");
Car c3 = new Car("BMW", "Black");
Car c4 = new Car("Tesla", "White", 90);

c1.status();  // Unknown | White  | fuel: 100% | 0km/h
c2.status();  // Toyota  | White  | fuel: 100% | 0km/h
c3.status();  // BMW     | Black  | fuel: 100% | 0km/h
c4.status();  // Tesla   | White  | fuel: 90%  | 0km/h
```

`this()` — Constructor Chaining

Repeating default values across constructors is messy. Use `this()` to call one constructor from another — eliminates duplication:

java

```
public class Car {
    String brand;
    String color;
    int fuelLevel;
    int speed;

    // Master constructor — all logic lives HERE
    public Car(String brand, String color, int fuelLevel) {
        this.brand     = brand;
        this.color     = color;
        this.fuelLevel = fuelLevel;
        this.speed     = 0;
        System.out.println("Car initialized: " + brand);
    }

    // Delegates to master constructor ↑
    public Car(String brand, String color) {
        this(brand, color, 100);       // calls 3-arg constructor
    }

    // Delegates to 2-arg constructor ↑
```

```
    public Car(String brand) {
        this(brand, "White");          // calls 2-arg constructor
    }

    // Delegates to 1-arg constructor ↑
    public Car() {
        this("Unknown");               // calls 1-arg constructor
    }
}
```

```
new Car()
    └──► this("Unknown")
             └──► this("Unknown", "White")
                      └──► this("Unknown", "White", 100)
                               └──► actual initialization runs
```

> `this()` **must be the first statement** in the constructor — no exceptions.

`this` keyword — all uses

`this` refers to the **current object instance**. Three main uses:

java

```java
public class Car {
    String brand;
    int speed;

    // USE 1 — distinguish field from parameter (same name)
    public Car(String brand) {
        this.brand = brand;   // this.brand = field, brand = parameter
    }

    // USE 2 — pass current object to another method
    public void register(CarRegistry registry) {
        registry.add(this);   // pass THIS car object to registry
    }

    // USE 3 — call another constructor (shown above)
    public Car() {
        this("Unknown");      // this() = constructor call
    }

    // USE 4 — return current object (enables method chaining)
    public Car accelerate(int amount) {
        this.speed += amount;
        return this;          // return the same object
```

```
    }
}
```

java

```java
// Method chaining thanks to return this
Car c = new Car("Tesla");
c.accelerate(30).accelerate(20).accelerate(10);
//  speed = 60 — each call returns the same object
```

Full mental model

```
new Car("Toyota", "Red", 100)
          |
          ▼
   Java allocates space on HEAP
          |
          ▼
   Matching constructor runs
   this.brand     = "Toyota"
   this.color     = "Red"
   this.fuelLevel = 100
   this.speed     = 0
          |
          ▼
   Reference returned → stored in variable
   Car c = [address → heap object]
```

Summary

Concept	Purpose
Default constructor	Create object with preset/zero values
Parameterized constructor	Create object with caller-supplied values
Constructor overloading	Flexible creation — multiple valid ways to construct
`this.field`	Refer to instance field when name conflicts with parameter
`this()`	Chain constructors — avoid duplicating initialization logic
`return this`	Enable fluent method chaining

> The golden rule: **an object should never exist in an invalid state.** Constructors are your guarantee.

Explain main method in Java and interview questions asked on it

The `main` Method in Java

The `main` method is the **entry point** of every Java application — it's where the JVM starts execution.

```java
public static void main(String[] args) {
    System.out.println("Hello, World!");
}
```

Anatomy — every keyword explained

```java
public static void main(String[] args)
|      |      |    |    └──────────────┘
|      |      |    |     command-line arguments
|      |      |    |     passed as String array
|      |      |    |
|      |      |    └── method name (must be exactly "main")
|      |      |
|      |      └── no return value
|      |          (JVM doesn't expect one back)
|      |
|      └── no object needed to call this
|          JVM calls it directly on the class
|
└── JVM comes from outside the class,
    so it must be accessible to everyone
```

Each keyword is **mandatory** — remove any one of them and the JVM won't recognize it as the entry point.

Why each keyword is required

`public`

The JVM is outside your class. It needs to **access** `main` from anywhere. Making it `public` ensures the JVM can call it without restriction.

```java
// private main — JVM can't see it
private static void main(String[] args) { }  // ❌ won't run
```

`static`

The JVM calls `main` **before creating any object**. Since `static` methods belong to the class (not an instance), the JVM can invoke it without doing `new MyClass()` first.

```java
// non-static main — JVM would need an object first
// but what object? who creates it? chicken-and-egg problem
public void main(String[] args) { }  // ❌ won't run
```

`void`

The JVM doesn't use a return value from `main`. When `main` returns, the program exits. If you want to exit with a status code, use `System.exit(code)` explicitly.

`main`

This is the **contract name** the JVM looks for by default. It's not a keyword — just a convention the JVM relies on.

`String[] args`

An array of `String` arguments passed from the **command line** at launch time. Can be empty but must be present in the signature.

Command-line arguments via `args`

```java
public class Greet {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("No name provided.");
        } else {
            System.out.println("Hello, " + args[0] + "!");
        }
    }
}
```

```bash
```

```
javac Greet.java
java Greet Alice        # Hello, Alice!
java Greet              # No name provided.
java Greet Alice Bob    # Hello, Alice!  (args[1] = "Bob" is ignored here)
```

`args` is just a regular `String[]` — you can loop it, parse it, convert types:

java

```java
public static void main(String[] args) {
    int sum = 0;
    for (String arg : args) {
        sum += Integer.parseInt(arg);   // convert String → int
    }
    System.out.println("Sum: " + sum);
}
```

bash

```
java Sum 10 20 30    # Sum: 60
```

Valid `main` signatures Java accepts

java

```java
// All of these are valid — JVM recognizes them
public static void main(String[] args)      // classic
public static void main(String args[])      // array syntax variant
static public void main(String[] args)      // modifier order doesn't matter
public static final void main(String[] args)    // final is allowed
public static synchronized void main(String[] args) // synchronized is allowed
public static void main(String... args)     // varargs — valid since Java 5
```

java

```java
// These will NOT work as entry points
public void main(String[] args)             // ❌ missing static
static void main(String[] args)             // ❌ missing public
public static int main(String[] args)       // ❌ wrong return type
public static void Main(String[] args)      // ❌ wrong name (case-sensitive)
public static void main()                   // ❌ missing args parameter
```

Can `main` be overloaded?

Yes — but the JVM only calls `main(String[] args)`:

java

```java
public class Demo {

    // JVM calls THIS one
    public static void main(String[] args) {
        System.out.println("Entry point");
        main(42);           // you can call overloaded version manually
    }

    // Overloaded — valid Java, but JVM won't call this directly
    public static void main(int number) {
        System.out.println("Overloaded main: " + number);
    }
}
```

```
Entry point
Overloaded main: 42
```

Can `main` be overridden?

No — `static` methods belong to the class, not an instance. They are **hidden**, not overridden, in subclasses:

java

```java
class Parent {
    public static void main(String[] args) {
        System.out.println("Parent main");
    }
}

class Child extends Parent {
    public static void main(String[] args) {
        System.out.println("Child main");     // this is method hiding, not overridi
    }
}
```

bash

```bash
java Parent    # Parent main
java Child     # Child main
```

Each class has its own independent `main` — the JVM picks based on which class you run.

`main` inside flow — what JVM does step by step

```
java MyApp
     |
     ▼
JVM starts up
     |
     ▼
Bootstrap ClassLoader loads java.lang.*
     |
     ▼
Application ClassLoader loads MyApp.class
     |
     ▼
JVM looks for: public static void main(String[])
     |
     ├── not found → error: Main method not found
     |
     └── found ──►  new thread created ("main" thread)
                          |
                          ▼
                    Stack frame for main() pushed
                          |
                          ▼
                    Your code runs
                          |
                          ▼
                    main() returns
                          |
                          ▼
                    JVM checks for non-daemon threads
                          |
                    none running → JVM exits
```

Interview Questions on `main`

Q1. Can you run a Java program without a `main` method?

In modern Java — **no.** But historically:

- Before Java 7 — you could run code via `static` blocks (JVM ran them then crashed with no main found, but output was visible)
- Java 7+ — JVM explicitly validates `main` exists before running static blocks

```
java
```

```
// This used to print then crash pre-Java 7
public class Trick {
    static {
        System.out.println("Static block ran!");
        System.exit(0);    // prevented the crash
    }
}
// Java 7+: Error: Main method not found — static block never runs
```

Q2. Why is `main` `static`?

Because the JVM needs to call it **without creating an object first**. There's no mechanism to tell the JVM which constructor to use or what arguments to pass — so `main` must be reachable at the class level via `static`.

Q3. What happens if you remove `String[] args`?

java

```
public static void main() { }   // ❌
```

The JVM looks for exactly `main(String[])`. This signature doesn't match — `Main method not found` error at runtime even though it compiles fine.

Q4. Can `main` be `final`?

java

```
public static final void main(String[] args) { }   // ✅ runs fine
```

Yes — `final` just prevents subclasses from hiding it (though since it's static, hiding was already not polymorphic). JVM doesn't care.

Q5. Can `main` throw exceptions?

java

```
public static void main(String[] args) throws Exception {   // ✅
    throw new Exception("Crash!");
}
```

Yes — if an uncaught exception escapes `main`, the JVM prints the stack trace and exits.

Q6. Can a class have multiple `main` methods?

Yes — through **overloading**. But the JVM only uses `main(String[] args)` as the entry point. Other `main` variants are just regular static methods.

Q7. Can `main` be `private` or `protected`?

```java
private static void main(String[] args) { }    // compiles but JVM can't call it
```

It compiles but the JVM requires `public` access to invoke it. You'll get `Main method not found` at runtime.

Q8. Is `main` a keyword in Java?

No. `main` is just an identifier — a regular method name that the JVM happens to look for by convention. You could name a variable `main`, or have a class method `main` that does something entirely different (though that's poor practice).

Q9. What is the return type of `main` and can you change it?

Return type must be `void`. The JVM doesn't collect any return value from `main`. If you want to signal an exit code to the OS, use:

```java
System.exit(0);    // success
System.exit(1);    // failure / abnormal exit
```

Q10. Can you call `main` explicitly from within a program?

Yes — it's just a static method. You can call it like any other:

```java
public class Demo {
    public static void main(String[] args) {
        System.out.println("main called");
        if (args.length == 0) {
            main(new String[]{"recursive"});   // call main explicitly
        }
    }
}
```

```
main called
main called
```

This won't cause infinite recursion here because of the `args.length` check — but it shows `main` is just a regular callable method.

Quick reference card

Aspect	Answer
Access modifier	Must be `public`
Static?	Must be `static`
Return type	Must be `void`
Method name	Must be exactly `main`
Parameter	Must be `String[]` or `String...`
Can be overloaded?	Yes
Can be overridden?	No (static → method hiding)
Can be `final`?	Yes
Can throw exceptions?	Yes
Is `main` a keyword?	No — just a convention

Access Modifiers
Packages & Imports

Access Modifiers & Packages in Java

Access Modifiers

Access modifiers control **who can see and use** a class, field, method, or constructor.

Java has four levels:

```
Most restrictive ◄──────────────────────────► Least restrictive
   private    →    default    →    protected    →    public
```

The four modifiers explained

`private` — class only

java

```java
public class BankAccount {
    private double balance;      // only THIS class can touch this
    private String pin;

    public BankAccount(double initialBalance, String pin) {
        this.balance = initialBalance;
        this.pin = pin;
    }

    private boolean validatePin(String input) {   // internal helper
        return this.pin.equals(input);
    }

    public void withdraw(String pin, double amount) {
        if (validatePin(pin)) {          // private method used internally
            balance -= amount;
        } else {
            System.out.println("Wrong PIN!");
        }
    }

    public double getBalance() {         // controlled access via public method
        return balance;
    }
}
```

java

```java
BankAccount acc = new BankAccount(5000, "1234");
acc.balance;          // ❌ compile error — private
acc.validatePin();    // ❌ compile error — private
acc.getBalance();     // ✅ works — public method exposes it safely
```

`default` (package-private) — no keyword written

When you write **no modifier**, access is limited to classes in the **same package**:

java

```java
// file: com/bank/Transaction.java
package com.bank;

class Transaction {              // no modifier = package-private
    double amount;               // package-private field
    String type;

    void process() {             // package-private method
        System.out.println("Processing " + type + ": " + amount);
    }
}
```

java

```java
// file: com/bank/BankAccount.java — SAME package ✅
package com.bank;

public class BankAccount {
    void makeTransaction() {
        Transaction t = new Transaction();   // ✅ same package
        t.process();                         // ✅ accessible
    }
}
```

java

```java
// file: com/app/Main.java — DIFFERENT package ❌
package com.app;
import com.bank.Transaction;     // ❌ compile error — not public
```

`protected` — package + subclasses

java

```java
// file: com/animals/Animal.java
package com.animals;

public class Animal {
    protected String name;
    protected int age;

    protected void breathe() {
        System.out.println(name + " is breathing");
    }
```

```java
    private void heartbeat() {       // only Animal itself
        System.out.println("heartbeat");
    }
}
```

java

```java
// file: com/animals/Dog.java — same package ✅
package com.animals;

public class Dog extends Animal {
    public void bark() {
        System.out.println(name + " barks!");     // ✅ protected field
        breathe();                                 // ✅ protected method
    }
}
```

java

```java
// file: com/pets/GuideDog.java — different package, but subclass ✅
package com.pets;
import com.animals.Animal;

public class GuideDog extends Animal {
    public void guide() {
        System.out.println(name + " is guiding");  // ✅ inherited protected
        breathe();                                  // ✅ inherited protected
    }
}
```

java

```java
// file: com/app/Main.java — different package, NOT a subclass ❌
package com.app;
import com.animals.Animal;

public class Main {
    public static void main(String[] args) {
        Animal a = new Animal();
        a.name = "Cat";      // ❌ protected — not a subclass here
        a.breathe();         // ❌ protected — not a subclass here
    }
}
```

`public` — everywhere

java

```java
package com.utils;

public class MathUtils {
    public static int add(int a, int b) {    // accessible from anywhere
        return a + b;
    }
}
```

java

```java
// from any package, any class
import com.utils.MathUtils;
int result = MathUtils.add(3, 5);    // ✅ works anywhere
```

Access modifier comparison table

Modifier	Same Class	Same Package	Subclass (diff pkg)	Everywhere
`private`	✅	❌	❌	❌
`default`	✅	✅	❌	❌
`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅

What can each modifier apply to?

Modifier	Class (top-level)	Field	Method	Constructor
`private`	❌	✅	✅	✅
`default`	✅	✅	✅	✅
`protected`	❌	✅	✅	✅
`public`	✅	✅	✅	✅

Top-level classes can only be `public` or package-private — never `private` or `protected`.

Packages

A **package** is a namespace that groups related classes together — like folders for your `.java` files.

```
src/
└── com/
    └── bank/
        ├── BankAccount.java     → package com.bank
        ├── Transaction.java     → package com.bank
        └── utils/
            └── CurrencyUtil.java → package com.bank.utils
```

Declaring a package

Must be the **first line** of every `.java` file:

```java
package com.bank;           // this file belongs to com.bank package

public class BankAccount {
    // ...
}
```

If you write no `package` declaration, the class goes into the **default (unnamed) package** — fine for quick experiments, bad practice in real projects.

Why use packages?

```
Without packages                With packages
────────────────                ─────────────
BankAccount.java                com.bank.BankAccount
Transaction.java                com.bank.Transaction
Date.java              →        com.bank.utils.Date
List.java                       com.collections.List
String.java                     java.lang.String

Name clashes everywhere!        No clash — full name is unique
```

Packages solve three problems: **organization, name collision, access control.**

Importing classes

java

```java
package com.app;

// Import a specific class
import com.bank.BankAccount;

// Import everything in a package (wildcard)
import com.bank.*;

// Static import — use static members without class name
import static java.lang.Math.sqrt;
import static java.lang.Math.PI;

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(5000, "1234");   // no prefix needed

        double result = sqrt(16);    // instead of Math.sqrt(16)
        double area = PI * 5 * 5;    // instead of Math.PI
    }
}
```

Import rules & behavior

java

```java
// ✅ specific import — preferred, clear what's being used
import java.util.ArrayList;
import java.util.HashMap;

// ✅ wildcard — imports ALL classes in java.util (not subpackages)
import java.util.*;

// ❌ wildcard does NOT import subpackages
import java.util.*;
// java.util.concurrent.CopyOnWriteArrayList still needs explicit import
import java.util.concurrent.CopyOnWriteArrayList;
```

`java.lang` **is always auto-imported** — that's why you can use `String`, `System`, `Math`, `Integer` without importing:

java

```java
// These are implicitly available everywhere
String s = "hello";          // java.lang.String
```

```
System.out.println(s);        // java.lang.System
Math.abs(-5);                 // java.lang.Math
```

Name collision — how to resolve

java

```
import java.util.Date;
import java.sql.Date;         // ❌ compile error — ambiguous
```

Solution — use **fully qualified name** for one of them:

java

```
import java.util.Date;        // import one

public class Main {
    public static void main(String[] args) {
        Date utilDate = new Date();                    // java.util.Date
        java.sql.Date sqlDate = new java.sql.Date(0); // fully qualified
    }
}
```

Package naming conventions

```
com.companyname.projectname.module

com.google.gson
com.amazon.s3.client
org.apache.commons.lang
java.util.concurrent

// Your projects
com.yourname.bankapp.accounts
com.yourname.bankapp.utils
```

Always **lowercase**, reversed domain name first — guarantees global uniqueness.

Full working example

```
src/
├── com/bank/
|   ├── BankAccount.java
```

```
|   └── utils/
|       └── CurrencyUtil.java
└── com/app/
    └── Main.java
```

java

```java
// com/bank/utils/CurrencyUtil.java
package com.bank.utils;

public class CurrencyUtil {
    public static String format(double amount) {
        return String.format("₹%.2f", amount);
    }
}
```

java

```java
// com/bank/BankAccount.java
package com.bank;

import com.bank.utils.CurrencyUtil;   // import from subpackage

public class BankAccount {
    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void showBalance() {
        System.out.println("Balance: " + CurrencyUtil.format(balance));
    }
}
```

java

```java
// com/app/Main.java
package com.app;

import com.bank.BankAccount;          // import from another package

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(5000.50);
        acc.showBalance();    // Balance: ₹5000.50
    }
}
```

Summary

```
Access Modifiers — control visibility
──────────────────────────────────────

private    → class only            (encapsulation)
default    → package only          (internal collaboration)
protected  → package + subclasses  (inheritance)
public     → everywhere            (API surface)

Packages — control organization
────────────────────────────────

package com.bank;          → declare membership
import com.bank.Account;   → use a specific class
import com.bank.*;         → use all classes in package
import static Math.sqrt;   → use static members directly
java.lang.*                → always auto-imported
```

> **Rule of thumb:** keep everything as restrictive as possible. Start with `private`, loosen only when needed. This is the foundation of **encapsulation** — a core OOP principle.

Data types — primitive (`int`, `char`, `boolean`, `double`, etc.)

Data Types in Java

Java is **statically typed** — every variable must have a declared type known at compile time. There are two categories:

```
Java Data Types
├── Primitive    — 8 types, store actual value, live on stack
└── Reference    — Objects, arrays, Strings — store address, live on heap
```

The 8 Primitive Types

Type	Size	Default	Range	Example literal
`byte`	1 byte	0	-128 to 127	`byte b = 100;`
`short`	2 bytes	0	-32,768 to 32,767	`short s = 3000;`

Type	Size	Default	Range	Example literal
`int`	4 bytes	0	-2.1B to 2.1B	`int i = 42;`
`long`	8 bytes	0L	-9.2Q to 9.2Q	`long l = 99L;`
`float`	4 bytes	0.0f	~7 decimal digits	`float f = 3.14f;`
`double`	8 bytes	0.0d	~15 decimal digits	`double d = 3.14;`
`char`	2 bytes	`\u0000`	0 to 65,535 (Unicode)	`char c = 'A';`
`boolean`	~1 bit*	`false`	`true` / `false`	`boolean flag = true;`

*JVM-dependent, not precisely defined in spec.

Integer types

java

```java
byte   b = 120;              // tiny — good for raw binary data
short  s = 30_000;           // rarely used
int    i = 2_147_483_647;    // most common — default for integers
long   l = 9_876_543_210L;   // needs L suffix — large numbers

// Underscores in literals (Java 7+) — improves readability
int million   = 1_000_000;
long cardNum  = 1234_5678_9012_3456L;

// Different bases
int decimal = 255;
int hex     = 0xFF;       // 255 — prefix 0x
int octal   = 0377;       // 255 — prefix 0
int binary  = 0b11111111; // 255 — prefix 0b (Java 7+)

System.out.println(decimal == hex);    // true — all are 255
```

Overflow — wraps around silently:

java

```java
int max = Integer.MAX_VALUE;     // 2147483647
System.out.println(max + 1);     // -2147483648  ← wraps around!

// Use long for big numbers
long bigNum = (long) max + 1;    // 2147483648 ✅
```

Floating point types

```java
float  f = 3.14f;        // needs f suffix, ~7 sig digits
double d = 3.14159265;   // default for decimals, ~15 sig digits

// Special values
double inf     = 1.0 / 0.0;          // Infinity
double negInf  = -1.0 / 0.0;         // -Infinity
double notNum  = 0.0 / 0.0;          // NaN (Not a Number)

System.out.println(inf);       // Infinity
System.out.println(notNum);    // NaN
System.out.println(Double.isNaN(notNum));      // true
System.out.println(Double.isInfinite(inf));    // true
```

⚠️ Floating point precision trap:

```java
double a = 0.1 + 0.2;
System.out.println(a);              // 0.30000000000000004  ← NOT 0.3!
System.out.println(a == 0.3);       // false  ← never compare with ==

// Fix 1 — round for display
System.out.printf("%.2f%n", a);     // 0.30

// Fix 2 — use BigDecimal for exact arithmetic (money, etc.)
import java.math.BigDecimal;
BigDecimal x = new BigDecimal("0.1");
BigDecimal y = new BigDecimal("0.2");
System.out.println(x.add(y));       // 0.3  ✅
```

> **Rule:** Never use `float` / `double` for money. Use `BigDecimal`.

`char` type

Stores a **single Unicode character** as a 16-bit unsigned integer:

```java
char letter  = 'A';          // single quotes — not double!
char digit   = '9';
char space   = ' ';
char unicode = '\u0041';     // 'A' — Unicode escape
char newline = '\n';         // escape sequence
```

```
// char is numeric underneath
char c = 'A';
System.out.println(c);         // A
System.out.println((int) c);   // 65  ← ASCII/Unicode value

// Arithmetic on chars
char next = (char) (c + 1);
System.out.println(next);      // B

// Iterating alphabet
for (char ch = 'a'; ch <= 'z'; ch++) {
    System.out.print(ch + " ");
}
// a b c d e f g h i j k l m n o p q r s t u v w x y z
```

Common escape sequences:

Sequence	Meaning
`'\n'`	newline
`'\t'`	tab
`'\\'`	backslash
`'\''`	single quote
`'\"'`	double quote
`'\u0041'`	Unicode (A)

`boolean` type

Only two values — `true` or `false`:

java

```
boolean isLoggedIn  = true;
boolean hasBalance  = false;

// Result of comparisons is boolean
boolean isAdult     = (age >= 18);
boolean isValid     = (balance > 0 && !isBlocked);

// Used in control flow
if (isLoggedIn && hasBalance) {
    System.out.println("Allow transaction");
}
```

```
// boolean cannot be cast to int (unlike C/C++)
int x = (int) true;    // ❌ compile error in Java
```

Type ranges — how to remember

```
byte   → 1 byte  → -2⁷   to 2⁷-1   →    -128 to 127
short  → 2 bytes → -2¹⁵  to 2¹⁵-1  → -32,768 to 32,767
int    → 4 bytes → -2³¹  to 2³¹-1  →      ~-2B to 2B
long   → 8 bytes → -2⁶³  to 2⁶³-1  →      ~-9Q to 9Q
```

Access them via constants:

java

```
System.out.println(Byte.MAX_VALUE);        //  127
System.out.println(Byte.MIN_VALUE);        // -128
System.out.println(Integer.MAX_VALUE);     //  2147483647
System.out.println(Long.MAX_VALUE);        //  9223372036854775807
System.out.println(Double.MAX_VALUE);      //  1.7976931348623157E308
```

Type casting

Two kinds — **widening** (automatic) and **narrowing** (manual):

```
byte → short → int → long → float → double
                                ↑
                          char ─┘
```

Widening — automatic, no data loss

java

```
int    i = 100;
long   l = i;        // ✅ int → long, automatic
double d = l;        // ✅ long → double, automatic
float  f = i;        // ✅ int → float, automatic

// char to int — widening
char c = 'A';
int  n = c;          // ✅ n = 65
```

Narrowing — manual cast required, may lose data

java

```
double d = 9.99;
int    i = (int) d;        // ✅ i = 9  — decimal part lost!

long   l = 1234567890123L;
int    x = (int) l;        // ✅ but value may be wrong — overflow risk

int    n = 65;
char   c = (char) n;       // ✅ c = 'A'

double price = 9.99;
float  f     = (float) price;   // some precision lost
```

Tricky casting cases

java

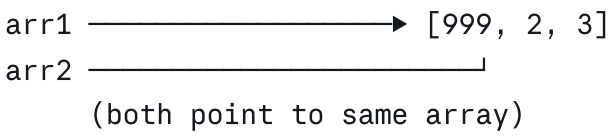
```
// int + int = int, even if stored in long
long result = 1_000_000 * 1_000_000;    // ❌ overflows int first!
long result = 1_000_000L * 1_000_000;   // ✅ forces long arithmetic

// byte arithmetic promotes to int automatically
byte a = 10, b = 20;
byte c = a + b;             // ❌ a+b produces int, needs cast
byte c = (byte)(a + b);     // ✅
```

Primitives vs Reference types

java

```
// Primitive — stores the VALUE directly
int a = 10;
int b = a;      // b gets a COPY of the value
b = 99;
System.out.println(a);   // 10 — a is unaffected

// Reference — stores an ADDRESS to the object
int[] arr1 = {1, 2, 3};
int[] arr2 = arr1;          // arr2 points to SAME array
arr2[0] = 999;
System.out.println(arr1[0]); // 999 — arr1 IS affected!
```

```
STACK                   HEAP
_____                   ____


a = 10                  (nothing — value is directly on stack)
b = 99
```



Wrapper classes

Every primitive has an **object wrapper** in `java.lang`:

Primitive	Wrapper
`int`	`Integer`
`long`	`Long`
`double`	`Double`
`float`	`Float`
`char`	`Character`
`boolean`	`Boolean`
`byte`	`Byte`
`short`	`Short`

Used when you need an object — collections, generics, nullability:

java

```java
// Autoboxing — primitive automatically wrapped into object
Integer obj = 42;           // int → Integer (automatic)

// Unboxing — object automatically unwrapped to primitive
int val = obj;              // Integer → int (automatic)

// Needed for collections (generics require objects, not primitives)
List<Integer> list = new ArrayList<>();
list.add(10);               // autoboxing: 10 → Integer(10)
int x = list.get(0);        // unboxing: Integer(10) → 10

// Useful methods on wrappers
Integer.parseInt("123");          // String → int
Integer.toBinaryString(255);      // "11111111"
Integer.toHexString(255);         // "ff"
Double.parseDouble("3.14");       // String → double
Character.isDigit('5');           // true
```

```
Character.isLetter('A');          // true
Character.toUpperCase('a');       // 'A'
```

Autoboxing trap — `==` vs `.equals()`:

java

```
Integer x = 127;
Integer y = 127;
System.out.println(x == y);       // true  (cached range: -128 to 127)

Integer p = 128;
Integer q = 128;
System.out.println(p == q);       // false (outside cache — different objects)
System.out.println(p.equals(q));  // true  ← always use equals() for wrappers
```

Summary

```
Integer types  →  byte  short  int(default)  long(L suffix)
Decimal types  →  float(f suffix)  double(default)
Text/char      →  char (single quotes, Unicode)
Logic          →  boolean (true/false only)

Widening  →  automatic  (small → large, no data loss)
Narrowing →  manual cast (large → small, possible data loss)

Primitives  →  value stored on stack, copied by value
Wrappers    →  object on heap, needed for collections/generics
BigDecimal  →  use for money/exact decimal arithmetic
```

Why do we need Wrapper Classes?
Difference between `int` and `Integer`?
Why use BigDecimal instead of double?

Wrapper Classes, int vs Integer, and BigDecimal

Why do we need Wrapper Classes?

Java's primitives (`int`, `double`, etc.) are **not objects** — they're raw values. But many parts of Java **require objects**. Wrapper classes bridge that gap.

Reason 1 — Collections only work with objects

java

```java
// ❌ primitives not allowed in generics
List<int>     list = new ArrayList<>();   // compile error
Map<int, int> map  = new HashMap<>();     // compile error

// ✅ wrapper classes work
List<Integer>        list = new ArrayList<>();
Map<String, Integer> map  = new HashMap<>();

list.add(10);     // int → Integer (autoboxing, automatic)
int x = list.get(0);  // Integer → int (unboxing, automatic)
```

Reason 2 — Nullability

A primitive **cannot be null** — it always has a value. A wrapper **can be null**, which is useful when absence of a value is meaningful:

java

```java
int     age = null;      // ❌ compile error — primitives can't be null
Integer age = null;      // ✅ means "age is unknown / not provided"

// Real-world use — database fields are often nullable
Integer salary = fetchFromDatabase("salary");   // might return null
if (salary != null) {
    System.out.println("Salary: " + salary);
} else {
    System.out.println("Salary not on record");
}
```

Reason 3 — Utility methods

Wrapper classes come packed with useful static methods:

java

```java
// Parsing strings → primitives
int    i = Integer.parseInt("123");
double d = Double.parseDouble("3.14");
long   l = Long.parseLong("9876543210");
boolean b = Boolean.parseBoolean("true");

// Number conversions
```

```java
String binary = Integer.toBinaryString(255);    // "11111111"
String hex    = Integer.toHexString(255);        // "ff"
String octal  = Integer.toOctalString(255);      // "377"

// Bounds
System.out.println(Integer.MAX_VALUE);    // 2147483647
System.out.println(Double.MIN_VALUE);     // 4.9E-324

// Comparison utilities
Integer.compare(5, 10);          // -1 (5 < 10)
Integer.max(5, 10);              // 10
Integer.sum(5, 10);              // 15
Integer.min(5, 10);              // 5

// Character utilities
Character.isDigit('5');          // true
Character.isLetter('A');         // true
Character.isWhitespace(' ');     // true
Character.toUpperCase('a');      // 'A'
Character.toLowerCase('Z');      // 'z'
```

Reason 4 — Generics & type parameters

java

```java
// Generic method needs a type — must be an object
public <T extends Comparable<T>> T findMax(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}

findMax(10, 20);          // ✅ Integer (autoboxed)
findMax(3.14, 2.71);      // ✅ Double  (autoboxed)
findMax("cat", "dog");    // ✅ String
```

Reason 5 — Reflection & Object-based APIs

java

```java
// Reflection requires objects
Object obj = 42;                          // autoboxed to Integer
System.out.println(obj.getClass());       // class java.lang.Integer

// Optional — wraps nullable values cleanly
Optional<Integer> maybeAge = Optional.of(25);
Optional<Integer> noAge    = Optional.empty();
```

`int` vs `Integer` — full comparison

	`int`	`Integer`
Type	Primitive	Object (reference)
Memory	4 bytes on stack	~16 bytes on heap + reference
Default value	`0`	`null`
Can be null?	❌ No	✅ Yes
In collections?	❌ No	✅ Yes
Has methods?	❌ No	✅ Yes
Comparison	`==` always works	`==` compares reference
Performance	Faster	Slower (object overhead)

The `==` trap with Integer

java

```
// int — == always compares values
int a = 1000;
int b = 1000;
System.out.println(a == b);       // true ✅

// Integer — == compares object references (addresses)
Integer x = 1000;
Integer y = 1000;
System.out.println(x == y);       // false ❌ — different objects on heap
System.out.println(x.equals(y));  // true  ✅ — always use equals()
```

Why does this work for small numbers though?

java

```
Integer p = 100;
Integer q = 100;
System.out.println(p == q);       // true  — wait, what?
```

Java caches `Integer` objects in the range **-128 to 127** (the `IntegerCache`). Within this range, `new Integer` is not created — the same cached object is returned. Outside this range, new objects are allocated.

java

```java
// Inside cache (-128 to 127) — same object returned
Integer a = 127;  Integer b = 127;
System.out.println(a == b);    // true  (same cached object)

// Outside cache — new objects
Integer c = 128;  Integer d = 128;
System.out.println(c == d);    // false (different objects)
System.out.println(c.equals(d)); // true ✅ — always use equals()
```

> **Rule:** Always use `.equals()` to compare `Integer` objects. Never `==`.

Autoboxing & Unboxing — the automatic conversion

java

```java
// Autoboxing — int → Integer (compiler inserts Integer.valueOf())
Integer obj = 42;
// compiler rewrites to: Integer obj = Integer.valueOf(42);

// Unboxing — Integer → int (compiler inserts .intValue())
int val = obj;
// compiler rewrites to: int val = obj.intValue();
```

Autoboxing trap — NullPointerException:

java

```java
Integer score = null;    // valid — Integer can be null
int    total  = score;   // ❌ NullPointerException!
                         // unboxing null calls null.intValue() → crash

// Always null-check before unboxing
if (score != null) {
    int total = score;
}
```

Autoboxing trap — performance in loops:

java

```java
// ❌ BAD — creates millions of Integer objects on heap
Long sum = 0L;
for (long i = 0; i < 1_000_000; i++) {
    sum += i;    // unbox sum, add i, rebox result — every iteration!
}
```

```
// ✅ GOOD — stays on stack, no object creation
long sum = 0L;
for (long i = 0; i < 1_000_000; i++) {
    sum += i;
}
```

Why use `BigDecimal` instead of `double`?

Because `double` uses **binary floating-point** (IEEE 754) — it cannot represent most decimal fractions exactly.

The core problem

java

```
System.out.println(0.1 + 0.2);          // 0.30000000000000004
System.out.println(1.03 - 0.42);        // 0.6099999999999999
System.out.println(0.1 * 3);            // 0.30000000000000004
System.out.println(4.0 / 3.0);          // 1.3333333333333333

// Money calculation with double — DANGEROUS
double price    = 19.99;
double tax      = 0.05;
double total    = price + (price * tax);
System.out.println(total);              // 20.9895 — should be 20.989500...
```

Why does this happen? `0.1` in binary is `0.000110011001100...` (repeating) — just like `1/3` in decimal is `0.333...`. It can never be stored exactly in a finite number of bits.

BigDecimal — exact decimal arithmetic

java

```
import java.math.BigDecimal;
import java.math.RoundingMode;

// Always use String constructor — not double!
BigDecimal a = new BigDecimal("0.1");
BigDecimal b = new BigDecimal("0.2");

System.out.println(a.add(b));           // 0.3  ✅ exact

// ❌ double constructor still has imprecision
```

```java
BigDecimal bad = new BigDecimal(0.1);
System.out.println(bad);   // 0.1000000000000000055511151231257827021181583404541015
```

BigDecimal operations

java

```java
BigDecimal price    = new BigDecimal("19.99");
BigDecimal taxRate  = new BigDecimal("0.05");
BigDecimal quantity = new BigDecimal("3");

// Arithmetic
BigDecimal tax      = price.multiply(taxRate);             // 0.9995
BigDecimal total    = price.add(tax);                      // 20.9895
BigDecimal subtotal = price.multiply(quantity);            // 59.97
BigDecimal discount = subtotal.subtract(new BigDecimal("5.00")); // 54.97
BigDecimal perItem  = subtotal.divide(quantity);           // 19.99

// Rounding — essential for money
BigDecimal rounded = total.setScale(2, RoundingMode.HALF_UP);
System.out.println(rounded);    // 20.99

// Comparison — use compareTo(), NOT equals()!
BigDecimal x = new BigDecimal("2.0");
BigDecimal y = new BigDecimal("2.00");
System.out.println(x.equals(y));        // false (scale differs!)
System.out.println(x.compareTo(y));     // 0  ✅ (numerically equal)
```

RoundingMode options

java

```java
BigDecimal n = new BigDecimal("2.345");

n.setScale(2, RoundingMode.HALF_UP);    // 2.35  — standard rounding
n.setScale(2, RoundingMode.HALF_DOWN);  // 2.34
n.setScale(2, RoundingMode.CEILING);    // 2.35  — always up
n.setScale(2, RoundingMode.FLOOR);      // 2.34  — always down
n.setScale(2, RoundingMode.UP);         // 2.35  — away from zero
n.setScale(2, RoundingMode.DOWN);       // 2.34  — toward zero
n.setScale(2, RoundingMode.HALF_EVEN);  // 2.34  — banker's rounding
```

Real-world money example

java

```java
import java.math.BigDecimal;
import java.math.RoundingMode;

public class Invoice {

    static BigDecimal applyTax(BigDecimal amount, BigDecimal taxRate) {
        return amount
            .multiply(taxRate)
            .setScale(2, RoundingMode.HALF_UP);
    }

    public static void main(String[] args) {
        BigDecimal price    = new BigDecimal("49.99");
        BigDecimal taxRate  = new BigDecimal("0.18");   // 18% GST
        BigDecimal quantity = new BigDecimal("4");

        BigDecimal subtotal = price.multiply(quantity);                 // 199.96
        BigDecimal tax      = applyTax(subtotal, taxRate);              //  35.99
        BigDecimal total    = subtotal.add(tax);                        // 235.95

        System.out.println("Subtotal : ₹" + subtotal);   // ₹199.96
        System.out.println("GST 18%  : ₹" + tax);        // ₹35.99
        System.out.println("Total    : ₹" + total);      // ₹235.95
    }
}
```

When to use what

```
int       → counting, indexing, general integers — default choice
Integer   → collections, nullability, parsing, generics
long      → large integers (timestamps, IDs, file sizes)
double    → scientific calculations where tiny errors are acceptable
float     → memory-sensitive bulk data (graphics, sensors)
BigDecimal→ money, tax, financial calculations — anywhere exactness matters
```

Summary

```
Wrapper classes needed for:
  ├── Collections / Generics   (List<Integer>, Map<String, Double>)
  ├── Nullability              (Integer can be null, int cannot)
  ├── Utility methods          (parseInt, toBinaryString, compare)
  └── Object-based APIs        (Reflection, Optional, streams)
```

```
int vs Integer:
  ├── int     → fast, stack, no null, no methods
  ├── Integer → object, heap, nullable, has methods
  └── ==  on Integer compares address — always use .equals()

double vs BigDecimal:
  ├── double     → fast, approximate, fine for science/display
  ├── BigDecimal → exact, slow, mandatory for money
  └── Always construct BigDecimal from String, never from double
```

Variables — local, instance, static
Explain with examples along with questions asked in interview

Variables in Java — Local, Instance, Static

A variable is a **named memory location** that holds a value. Where you declare it determines its **scope, lifetime, default value, and memory location.**

Three types at a glance

```
class BankAccount {

    static int totalAccounts = 0;    // ← STATIC variable
    String owner;                    // ← INSTANCE variable
    double balance;                  // ← INSTANCE variable

    void deposit(double amount) {
        double newBalance = balance + amount;  // ← LOCAL variable
        balance = newBalance;
    }
}
```

---

1. Local Variables

Declared **inside a method, constructor, or block.** Exist only while that block executes.

java

```
public class Calculator {

    public int add(int a, int b) {
```

```
        int result = a + b;       // local variable
        return result;
    }                             // result dies here

    public void process() {
        int x = 10;               // local to process()

        if (x > 5) {
            int y = 20;           // local to this if-block
            System.out.println(x + y);   // ✅ both visible here
        }

        System.out.println(x);    // ✅ x still visible
        System.out.println(y);    // ❌ compile error — y out of scope
    }
}
```

Key rules:

- **No default value** — must be initialized before use
- **No access modifiers** — can't be `public`, `private`, etc.
- **Lives on the stack** — created when block enters, destroyed when block exits

java

```
public void demo() {
    int x;                     // declared but NOT initialized
    System.out.println(x);    // ❌ compile error: variable x might not be initialize

    int y = 0;
    System.out.println(y);    // ✅ 0
}
```

---

2. Instance Variables

Declared **inside the class but outside any method.** Each object gets its **own independent copy.**

java

```
public class Student {

    // instance variables — each Student object has its own copy
    String name;
    int    age;
    double gpa;
```

```java
    public Student(String name, int age, double gpa) {
        this.name = name;
        this.age  = age;
        this.gpa  = gpa;
    }

    public void display() {
        System.out.println(name + " | Age: " + age + " | GPA: " + gpa);
    }
}
```

java

```java
Student s1 = new Student("Alice", 20, 3.9);
Student s2 = new Student("Bob",   22, 3.5);
Student s3 = new Student("Carol", 21, 3.7);

s1.display();    // Alice | Age: 20 | GPA: 3.9
s2.display();    // Bob   | Age: 22 | GPA: 3.5

// Each object owns its data — changing one doesn't affect others
s1.gpa = 4.0;
System.out.println(s2.gpa);    // 3.5 — unaffected
```

Memory:

```
HEAP
————
s1 → [ name="Alice", age=20, gpa=4.0 ]
s2 → [ name="Bob",   age=22, gpa=3.5 ]
s3 → [ name="Carol", age=21, gpa=3.7 ]
```

Default values when not initialized:

Type	Default
`int`, `short`, `byte`, `long`	`0`
`float`, `double`	`0.0`
`boolean`	`false`
`char`	`'\u0000'`
Object reference	`null`

java

```java
public class Demo {
    int     count;     // defaults to 0
    double  price;     // defaults to 0.0
    boolean active;    // defaults to false
    String  name;      // defaults to null

    void show() {
        System.out.println(count);    // 0
        System.out.println(price);    // 0.0
        System.out.println(active);   // false
        System.out.println(name);     // null
    }
}
```

---

3. Static Variables

Declared with `static` keyword. **One copy shared across ALL objects** of the class. Belongs to the class itself, not any instance.

java

```java
public class BankAccount {

    // static — ONE value shared by all BankAccount objects
    static int    totalAccounts = 0;
    static String bankName      = "JavaBank";

    // instance — each account has its own
    String owner;
    double balance;
    int    accountNumber;

    public BankAccount(String owner, double initialBalance) {
        totalAccounts++;                    // shared counter goes up
        this.accountNumber = totalAccounts; // assign current count as ID
        this.owner         = owner;
        this.balance       = initialBalance;
    }

    public static void showBankInfo() {     // static method
        System.out.println(bankName + " | Total Accounts: " + totalAccounts);
    }

    public void showAccount() {
        System.out.println("Acct #" + accountNumber + " | " + owner + " | ₹" + balan
```

```
    }
}
```

java

```
BankAccount a1 = new BankAccount("Alice", 50_000);
BankAccount a2 = new BankAccount("Bob",   30_000);
BankAccount a3 = new BankAccount("Carol", 75_000);

a1.showAccount();          // Acct #1 | Alice | ₹50000.0
a2.showAccount();          // Acct #2 | Bob   | ₹30000.0
a3.showAccount();          // Acct #3 | Carol | ₹75000.0

BankAccount.showBankInfo(); // JavaBank | Total Accounts: 3
```

Memory:

```
METHOD AREA (class-level)
──────────────────────────
BankAccount.totalAccounts = 3
BankAccount.bankName      = "JavaBank"

HEAP (per object)
──────────────────────────────────────────
a1 → [ owner="Alice", balance=50000, accountNumber=1 ]
a2 → [ owner="Bob",   balance=30000, accountNumber=2 ]
a3 → [ owner="Carol", balance=75000, accountNumber=3 ]
     (all three share the static variables above)
```

Side-by-side comparison

java

```
public class Employee {

    static String company   = "TechCorp";  // STATIC  — shared by all
    String        name;                    // INSTANCE — per object
    double        salary;                  // INSTANCE — per object

    public Employee(String name, double salary) {
        this.name   = name;
        this.salary = salary;
    }

    public void review() {
        double bonus = salary * 0.10;      // LOCAL — lives only here
```

```
        System.out.println(name + " bonus: ₹" + bonus);
    }
}
```

	Local	Instance	Static
Declared in	Method/block	Class body	Class body with `static`
Belongs to	Block execution	Each object	The class
Memory	Stack	Heap	Method Area
Lifetime	Block duration	Object lifetime	Program lifetime
Default value	❌ None (must init)	✅ Yes (0/null/false)	✅ Yes (0/null/false)
Access modifier	❌ Not allowed	✅ Allowed	✅ Allowed
Access via	Variable name	`object.field`	`ClassName.field`

Static variable — shared state in action

java

```java
public class Counter {
    static int count = 0;   // shared

    public Counter() {
        count++;
    }

    public static void main(String[] args) {
        System.out.println(Counter.count);   // 0

        new Counter();
        new Counter();
        new Counter();

        System.out.println(Counter.count);   // 3 — all three bumped same variable
    }
}
```

Shadowing — local hides instance

java

```java
public class Shadow {
    int x = 100;            // instance variable

    public void show(int x) {   // parameter shadows instance variable
        System.out.println(x);        // prints parameter value
        System.out.println(this.x);   // this.x explicitly accesses instance variabl
    }

    public static void main(String[] args) {
        new Shadow().show(999);
        // 999   ← local parameter
        // 100   ← instance variable via this.x
    }
}
```

Interview Questions

Q1. What is the difference between instance and static variables?

Instance variables have a **separate copy per object** — each object owns its own value. Static variables have **one copy for the entire class** — all objects read and write the same memory location. Instance variables live on the heap inside each object; static variables live in the Method Area.

```java
class Car {
    static int wheels = 4;   // all cars have 4 wheels — shared fact
    String     color;        // each car has its own color — per object
}
```

Q2. What is the default value of a local variable?

None. Unlike instance and static variables, local variables are **never given a default value** by the JVM. The compiler enforces that you initialize them before use — reading an uninitialized local is a **compile-time error**.

```java
void demo() {
    int x;
    System.out.println(x);    // ❌ compile error
}
```

Q3. Can you access a non-static variable inside a static method?

No — directly. A static method runs without any object, so there is no `this` and no instance to read fields from. You must pass an object reference:

```java
class Demo {
    int value = 10;

    static void show() {
        System.out.println(value);          // ❌ compile error

        Demo d = new Demo();
        System.out.println(d.value);        // ✅ via an object reference
    }
}
```

Q4. Can you access a static variable inside an instance method?

Yes — always. Instance methods have access to both instance and static members:

```java
class Demo {
    static int count = 5;
    int id = 1;

    void show() {
        System.out.println(count);   // ✅ static accessible in instance method
        System.out.println(id);      // ✅ instance variable
    }
}
```

Q5. Where are local, instance, and static variables stored in memory?

```
Local    → Stack  (inside the method's stack frame)
Instance → Heap   (inside the object)
Static   → Method Area  (class-level memory, loaded once per class)
```

Q6. What happens to static variables when objects are created and destroyed?

Q7. What is variable shadowing? How do you fix it?

Shadowing occurs when a local variable (or parameter) has the **same name as an instance variable**, hiding it inside that scope. Fix it using `this.variableName` to explicitly refer to the instance variable:

```java
class Box {
    int width = 10;

    void resize(int width) {        // parameter shadows instance field
        System.out.println(width);       // parameter: whatever was passed
        System.out.println(this.width);  // instance field: 10
        this.width = width;              // fix: assign param to instance field
    }
}
```

Q8. Can a static variable be declared inside a method?

No. `static` is a class-level concept. You cannot declare a static variable inside a method — it's a compile error:

```java
void demo() {
    static int x = 10;    // ❌ compile error — illegal modifier
}
```

Q9. What is the output?

```java
class Test {
    static int x = 5;

    public static void main(String[] args) {
        Test a = new Test();
        Test b = new Test();

        a.x = 10;
        System.out.println(b.x);    // ?
```

```
    }
}
```

Output: `10` — `x` is static, so `a.x` and `b.x` refer to the **same variable**. Changing it via `a` is visible via `b`. (Note: accessing static via instance reference like `a.x` is poor style — prefer `Test.x`.)

Q10. Can two local variables in the same method have the same name?

No — if they are in the same or overlapping scope. But a variable in an inner block can shadow one in an outer block in some cases:

java

```java
void demo() {
    int x = 10;
    int x = 20;       // ❌ compile error — duplicate in same scope

    int y = 10;
    {
        int y = 20;   // ❌ still compile error in Java — unlike C/C++
    }
}
```

Java is stricter than C/C++ — you **cannot** shadow a local variable with another local variable in a nested block.

Full summary

```
LOCAL
  ├── Inside method/block/constructor
  ├── Stack memory
  ├── No default — MUST initialize
  ├── No access modifiers
  └── Scope ends when block ends

INSTANCE
  ├── Inside class, outside methods
  ├── Heap memory (inside object)
  ├── Default values (0, null, false)
  ├── Access modifiers allowed
  └── One copy per object

STATIC
  ├── Inside class with static keyword
  ├── Method Area memory (class-level)
```

```
├── Default values (0, null, false)
├── Access modifiers allowed
├── One copy for ALL objects
└── Access via ClassName.variable
```

Constants (`final` keyword)

Constants & the `final` Keyword in Java

`final` is a **non-access modifier** that means **"this cannot be changed after assignment"**. It applies to variables, methods, and classes — with different meaning in each case.

`final` Variables — Constants

Once assigned, the value **cannot be reassigned:**

java

```java
public class Circle {

    final double PI = 3.14159265358979;   // constant

    public double area(double radius) {
        PI = 3.0;    // ❌ compile error: cannot assign a value to final variable PI
        return PI * radius * radius;    // ✅ reading is fine
    }
}
```

Three kinds of `final` variables

---

1. Final Local Variable

java

```java
public void process() {
    final int MAX_RETRY = 3;     // assigned once, never changed

    MAX_RETRY = 5;    // ❌ compile error

    for (int i = 0; i < MAX_RETRY; i++) {
```

```
        System.out.println("Attempt " + (i + 1));
    }
}
```

A final local variable doesn't need to be initialized at declaration — but must be initialized **before first use**, exactly once:

java

```
public void demo(boolean flag) {
    final int x;            // declared but not yet initialized

    if (flag) {
        x = 10;             // ✅ assigned in one branch
    } else {
        x = 20;             // ✅ assigned in other branch
    }

    System.out.println(x);  // ✅ safe — guaranteed to be assigned
    x = 99;                 // ❌ compile error — already assigned
}
```

---

2. Final Instance Variable

Must be initialized either at declaration or **inside the constructor** — never after:

java

```
public class Person {

    final String name;          // not initialized here
    final String nationalId;    // not initialized here
    final int    birthYear = 1995;  // initialized at declaration ✅

    public Person(String name, String nationalId) {
        this.name       = name;          // ✅ assigned in constructor
        this.nationalId = nationalId;    // ✅ assigned in constructor
    }

    public void birthday() {
        name = "New Name";    // ❌ compile error — already set
    }
}
```

java

```
Person p = new Person("Alice", "ID-001");
```

```
p.name = "Bob";    // ❌ compile error — final
```

---

3. Final Static Variable (true constants)

The most common pattern — `static final` together:

java

```java
public class MathConstants {

    // Convention: UPPER_SNAKE_CASE for constants
    public static final double PI          = 3.14159265358979;
    public static final double E           = 2.71828182845904;
    public static final int    MAX_INT     = Integer.MAX_VALUE;
    public static final String APP_VERSION = "2.1.0";
}
```

java

```java
// Access via class name — no object needed
double area = MathConstants.PI * radius * radius;
System.out.println(MathConstants.APP_VERSION);    // 2.1.0
```

Static final must be initialized at declaration or in a static block:

java

```java
public class Config {

    public static final String DB_URL;
    public static final int    TIMEOUT;

    // Static initializer block — runs once when class loads
    static {
        DB_URL  = "jdbc:mysql://localhost:3306/mydb";
        TIMEOUT = 30;
    }
}
```

`final` with Reference types — the important trap

`final` on a reference means the **reference cannot point to a different object** — but the object's contents can still change:

java

```java
```

```java
public class Demo {
    public static void main(String[] args) {

        final int[] numbers = {1, 2, 3};

        numbers[0] = 99;          // ✅ modifying contents — allowed
        System.out.println(numbers[0]);    // 99

        numbers = new int[]{4, 5, 6};     // ❌ compile error — reassigning referenc

        // ──────────────────────────────────────

        final StringBuilder sb = new StringBuilder("Hello");

        sb.append(" World");      // ✅ modifying object — allowed
        System.out.println(sb);   // Hello World

        sb = new StringBuilder(); // ❌ compile error — reassigning reference
    }
}
```

Mental model:

```
final int[] arr = {1, 2, 3}

STACK           HEAP
─────           ────
arr ──────────► [1, 2, 3]    ← contents CAN change
 ↑
 └── this arrow is final — cannot point elsewhere
```

Real-world constant patterns

App-wide constants class

java

```java
public class AppConstants {

    // prevent instantiation
    private AppConstants() { }

    // Server config
    public static final String HOST        = "api.myapp.com";
    public static final int    PORT        = 8080;
    public static final int    TIMEOUT_MS  = 5000;
```

```
    // Business rules
    public static final int    MAX_LOGIN_ATTEMPTS = 5;
    public static final double GST_RATE           = 0.18;
    public static final int    SESSION_EXPIRY_MIN = 30;

    // Error messages
    public static final String ERR_NOT_FOUND   = "Resource not found";
    public static final String ERR_UNAUTHORIZED = "Access denied";
}
```

java

```
// Use anywhere in the project
if (loginAttempts > AppConstants.MAX_LOGIN_ATTEMPTS) {
    System.out.println(AppConstants.ERR_UNAUTHORIZED);
}

double tax = amount * AppConstants.GST_RATE;
```

`final` Methods — cannot be overridden

java

```
public class Payment {

    // subclasses cannot override this — security-critical logic
    public final void processPayment(double amount) {
        validate(amount);
        deductBalance(amount);
        logTransaction(amount);
    }

    // these CAN be overridden
    protected void validate(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Invalid amount");
    }
}
```

java

```
public class UPIPayment extends Payment {

    @Override
    public void processPayment(double amount) {    // ❌ compile error
        // cannot override final method
    }
```

```
    @Override
    protected void validate(double amount) {       // ✅ not final, can override
        // UPI-specific validation
    }
}
```

When to use: Security-critical methods, template method patterns where the overall flow must stay fixed.

`final` Classes — cannot be extended

java

```
public final class ImmutablePoint {
    private final int x;
    private final int y;

    public ImmutablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }
}
```

java

```
public class MutablePoint extends ImmutablePoint {    // ❌ compile error
    // cannot extend a final class
}
```

Famous final classes in Java:

java

```
public final class String   { }     // java.lang.String
public final class Integer  { }     // java.lang.Integer
public final class Math     { }     // java.lang.Math
```

`String` is `final` — no one can subclass it and break its immutability contract.

`final` + `static` + `private` — full encapsulation

java

```java
public class DatabaseConfig {

    private static final String URL      = "jdbc:mysql://localhost/db";
    private static final String USER     = "admin";
    private static final String PASSWORD = "secret";
    private static final int    POOL     = 10;

    private DatabaseConfig() { }   // no instances

    public static String getUrl()  { return URL; }
    public static int    getPool() { return POOL; }
    // PASSWORD intentionally not exposed
}
```

`final` in summary — three contexts

java

```java
// 1. final VARIABLE — value cannot be reassigned
final int x = 10;
x = 20;    // ❌

// 2. final METHOD — cannot be overridden in subclass
public final void pay() { }

// 3. final CLASS — cannot be subclassed
public final class MyClass { }
```

Blank final variable

A `final` variable declared **without initialization** — must be assigned exactly once before use:

java

```java
public class Order {
    final String orderId;      // blank final — not yet set

    public Order(String id) {
        orderId = id;          // ✅ assigned in constructor
    }

    public Order() {
        // orderId not assigned   ❌ compile error
        // — blank final must be assigned in every constructor path
```

```
    }
}
```

Interview questions

Q1. Can a `final` variable be left uninitialized?

For **local** variables — yes, if initialized before first use. For **instance** final variables — must be initialized at declaration or in every constructor. For **static final** — at declaration or in a static block.

Q2. Does `final` make an object immutable?

No. `final` on a reference only prevents the reference from pointing to a different object. The object's internal state can still be mutated unless the class is designed to be immutable (all fields private + final, no setters):

java

```java
final List<String> list = new ArrayList<>();
list.add("hello");    // ✅ object mutated — final doesn't stop this
list = new ArrayList<>();  // ❌ reference reassignment — stopped by final
```

Q3. Can we initialize a `final` instance variable in a method?

No — only at declaration or in a constructor. Assigning in a regular method is a compile error:

java

```java
class Demo {
    final int x;

    void init() {
        x = 10;    // ❌ compile error — too late, not a constructor
    }
}
```

Q4. What is the difference between `final`, `finally`, and `finalize`?

```
final      → keyword: prevents reassignment / override / inheritance
finally    → try-catch block: code that always runs after try/catch
```

```
finalize() → deprecated method: called by GC before object is collected
```

```java
// final — modifier
final int x = 10;

// finally — exception handling
try {
    riskyOperation();
} catch (Exception e) {
    handle(e);
} finally {
    cleanup();    // always runs
}

// finalize — Object method (deprecated since Java 9)
@Override
protected void finalize() {
    // cleanup before GC — unreliable, avoid using
}
```

Q5. Why is `String` a `final` class?

Three reasons: **security** (no subclass can override `String` behavior and sneak malicious data into APIs expecting strings), **immutability** (the whole design of `String` relies on its value never changing — caching, hash codes, thread safety), and **performance** (the JVM heavily optimizes `String` because it knows exactly what it is).

Q6. Can a `final` method be overloaded?

Yes — overloading and overriding are different things. `final` blocks **overriding** (same signature in subclass), but you can freely **overload** (different parameters) even in the same class:

```java
public final void pay(double amount) { }      // final
public void pay(double amount, String upi) { } // ✅ overloaded — different params
```

Quick reference

Applied to	Effect
`final` local variable	Cannot be reassigned after first assignment
`final` instance variable	Must be set in constructor, never changed after
`final` static variable	Class-level constant, initialized once
`final` method	Cannot be overridden in subclasses
`final` class	Cannot be subclassed at all
`final` reference	Reference is fixed; object contents can still change

> **Rule of thumb:** use `final` by default for anything that shouldn't change. It documents intent, enables compiler optimizations, and prevents accidental mutation bugs.

Operators — arithmetic, logical, bitwise, ternary, shift
Most common Interview Questions asked

Operators in Java

An operator takes one or more operands and produces a result. Java has 6 major categories.

---

1. Arithmetic Operators

Basic math operations:

```java
int a = 17, b = 5;

System.out.println(a + b);   // 22  — addition
System.out.println(a - b);   // 12  — subtraction
System.out.println(a * b);   // 85  — multiplication
System.out.println(a / b);   // 3   — integer division (truncates!)
System.out.println(a % b);   // 2   — modulus (remainder)
```

Integer division truncates — does not round:

```java
```

```
System.out.println(7 / 2);       // 3   not 3.5
System.out.println(7.0 / 2);     // 3.5 — one double forces double math
System.out.println((double)7/2); // 3.5 — cast first, then divide
```

Modulus — not just for integers:

java

```
System.out.println(17 % 5);      // 2
System.out.println(-17 % 5);     // -2  — sign follows LEFT operand
System.out.println(17 % -5);     // 2
System.out.println(5.5 % 2.1);   // 1.3 — works on doubles too
```

Increment & Decrement:

java

```
int x = 5;

// Prefix — increment THEN use
System.out.println(++x);    // 6  (x becomes 6, then printed)
System.out.println(x);      // 6

// Postfix — use THEN increment
int y = 5;
System.out.println(y++);    // 5  (printed first, then y becomes 6)
System.out.println(y);      // 6
```

The classic trap:

java

```
int a = 5;
int b = a++ + ++a;
//      5   +  7    =  12
//      ↑ post: use 5, a→6
//             ↑ pre: a→7, use 7
System.out.println(b);    // 12
System.out.println(a);    // 7
```

---

2. Relational (Comparison) Operators

Always return `boolean`:

java

```
int a = 10, b = 20;
```

```
System.out.println(a == b);   // false — equal to
System.out.println(a != b);   // true  — not equal
System.out.println(a >  b);   // false — greater than
System.out.println(a <  b);   // true  — less than
System.out.println(a >= b);   // false — greater than or equal
System.out.println(a <= b);   // true  — less than or equal
```

`==` on objects — reference, not value:

java

```
String s1 = new String("hello");
String s2 = new String("hello");

System.out.println(s1 == s2);        // false — different objects on heap
System.out.println(s1.equals(s2));   // true  — same content

// String pool literals — same reference
String s3 = "hello";
String s4 = "hello";
System.out.println(s3 == s4);        // true  — same pool object
```

---

3. Logical Operators

Combine boolean expressions:

java

```
boolean a = true, b = false;

System.out.println(a && b);    // false — AND: both must be true
System.out.println(a || b);    // true  — OR:  at least one true
System.out.println(!a);        // false — NOT: inverts
```

Short-circuit evaluation — critical behavior:

java

```
// && stops at first false
int x = 0;
if (x != 0 && (10 / x > 1)) {     // second part NEVER evaluated
    System.out.println("yes");
}
// no ArithmeticException — && short-circuits at x != 0 (false)

// || stops at first true
String s = null;
if (s == null || s.isEmpty()) {    // second part NEVER evaluated if s==null
```

```
    System.out.println("empty");
}
// no NullPointerException — || short-circuits at s == null (true)
```

Non-short-circuit operators `&` and `|`:

java

```
int x = 0;
if (x != 0 & (10 / x > 1)) {   // ❌ both sides ALWAYS evaluated
    // throws ArithmeticException — division by zero
}
```

```
&&  →  short-circuit AND  — stops early, use this almost always
||  →  short-circuit OR   — stops early, use this almost always
&   →  non-short-circuit AND  — both sides always evaluated
|   →  non-short-circuit OR   — both sides always evaluated
```

---

4. Bitwise Operators

Operate on **individual bits** of integers:

java

```
int a = 12;   // binary: 1100
int b = 10;   // binary: 1010

//         1100
//         1010
//         ————
// AND  →  1000  = 8   (bit set only if BOTH bits are 1)
// OR   →  1110  = 14  (bit set if EITHER bit is 1)
// XOR  →  0110  = 6   (bit set if bits are DIFFERENT)
// NOT  → ~1100  = -13 (flips all bits, including sign bit)

System.out.println(a & b);    //  8
System.out.println(a | b);    // 14
System.out.println(a ^ b);    //  6
System.out.println(~a);       // -13
```

Visualized bit by bit:

```
a  =  0000 1100   (12)
b  =  0000 1010   (10)

a & b = 0000 1000  =  8   (AND  — 1 only where both are 1)
a | b = 0000 1110  = 14   (OR   — 1 where either is 1)
```

```
a ^ b = 0000 0110  =  6   (XOR  — 1 where they differ)
~a    = 1111 0011  = -13  (NOT  — all bits flipped)
```

Practical use — check/set/clear/toggle bits:

java

```java
int flags = 0b0000;

// Set bit 2 (turn ON)
flags = flags | (1 << 2);       // flags = 0100 = 4

// Check bit 2 (is it ON?)
boolean isSet = (flags & (1 << 2)) != 0;   // true

// Clear bit 2 (turn OFF)
flags = flags & ~(1 << 2);      // flags = 0000 = 0

// Toggle bit 2 (flip)
flags = flags ^ (1 << 2);       // flags = 0100 = 4 again
```

XOR swap — no temp variable:

java

```java
int x = 5, y = 9;
x = x ^ y;    // x = 5^9 = 12
y = x ^ y;    // y = 12^9 = 5  (original x)
x = x ^ y;    // x = 12^5 = 9  (original y)
System.out.println(x + " " + y);   // 9 5
```

---

5. Shift Operators

Shift bits left or right — equivalent to fast multiply/divide by powers of 2:

java

```java
int a = 8;    // binary: 0000 1000

// Left shift  << n  →  multiply by 2ⁿ
System.out.println(a << 1);    // 16  (8 × 2¹)
System.out.println(a << 2);    // 32  (8 × 2²)
System.out.println(a << 3);    // 64  (8 × 2³)

// Signed right shift >> n  →  divide by 2ⁿ (preserves sign)
System.out.println(a >> 1);    // 4   (8 / 2¹)
System.out.println(a >> 2);    // 2   (8 / 2²)
```

```
// Unsigned right shift >>> n  →  fills with 0 (ignores sign)
int neg = -8;
System.out.println(neg >> 1);   // -4  (sign preserved)
System.out.println(neg >>> 1);  // 2147483644  (fills with 0, huge positive)
```

Visualized:

```
a = 0000 1000  (8)

a << 1  →  0001 0000  = 16   (shift left,  fill right with 0)
a >> 1  →  0000 0100  =  4   (shift right, fill left with sign bit)

neg = 1111 1000  (-8)
neg >>  1  →  1111 1100  = -4           (sign bit (1) copied in)
neg >>> 1  →  0111 1100  = 2147483644   (0 filled in — not sign bit)
```

Why shift instead of multiply?

java

```
// These are equivalent but shift is marginally faster at hardware level
int x = 5;
System.out.println(x * 4);    // 20
System.out.println(x << 2);   // 20  — 5 × 2² = 20

System.out.println(x / 8);    // 0  (int division)
System.out.println(x >> 3);   // 0  — 5 / 2³ = 0

// Common in hashing, encoding, graphics, low-level protocols
int rgb   = (255 << 16) | (128 << 8) | 64;   // pack R,G,B into one int
int red   = (rgb >> 16) & 0xFF;               // extract red channel
int green = (rgb >> 8)  & 0xFF;               // extract green channel
int blue  =  rgb        & 0xFF;               // extract blue channel
```

---

6. Ternary Operator

Compact `if-else` in a single expression:

java

```
// Syntax: condition ? valueIfTrue : valueIfFalse

int age    = 20;
String result = (age >= 18) ? "Adult" : "Minor";
System.out.println(result);    // Adult

// Nested ternary — readable if kept simple
```

```java
int marks = 75;
String grade = (marks >= 90) ? "A"
             : (marks >= 75) ? "B"
             : (marks >= 60) ? "C"
             : "F";
System.out.println(grade);     // B
```

Ternary in expressions:

java

```java
int a = 10, b = 20;
int max = (a > b) ? a : b;                // 20
System.out.println("Max: " + max);

// Inline — avoid if too complex
System.out.println("Fee: ₹" + (isPremium ? 0 : 499));
```

Operator Precedence

Higher up = evaluated first:

```
Priority   Operators
────────   ──────────────────────────────
  1        ++  --  (postfix)
  2        ++  --  !  ~  (prefix/unary)
  3        *   /   %
  4        +   -
  5        <<  >>  >>>
  6        <   >   <=  >=  instanceof
  7        ==  !=
  8        &
  9        ^
  10       |
  11       &&
  12       ||
  13       ?:  (ternary)
  14       =   +=  -=  *=  /=  etc.
```

java

```java
int result = 2 + 3 * 4;          // 14 not 20 — * before +
int result = (2 + 3) * 4;        // 20 — parentheses override
boolean x  = true || false && false;  // true — && before ||
```

> **Rule:** When in doubt, use **parentheses**. Code clarity beats memorizing precedence.

Interview Questions

Q1. What is the output?

```java
int x = 10;
System.out.println(x++);   // ?
System.out.println(++x);   // ?
```

```
10  — postfix: print first (10), then increment (x=11)
12  — prefix:  increment first (x=12), then print
```

Q2. What is the output?

```java
int a = 5, b = 3;
System.out.println(a++ + ++b + a);
```

```
Step by step:
a++  → use 5, then a becomes 6
++b  → b becomes 4, use 4
a    → a is now 6
5 + 4 + 6 = 15
```

Q3. Difference between `&` and `&&`?

`&&` is short-circuit — if left side is `false`, right side is **never evaluated**. `&` always evaluates both sides. In practice, always use `&&` for boolean logic to avoid null pointer or divide-by-zero errors on the right side.

Q4. What is the output?

```java
System.out.println(10 / 3);      // ?
System.out.println(10 / 3.0);    // ?
System.out.println(10 % 3);      // ?
```

Q5. How do you check if a number is even using bitwise?

java

```java
// Bitwise AND with 1 — checks last bit
// Even numbers always have last bit = 0
// Odd  numbers always have last bit = 1

int n = 14;
if ((n & 1) == 0) {
    System.out.println("Even");   // Even
} else {
    System.out.println("Odd");
}
```

Faster than `n % 2 == 0` at the hardware level.

Q6. What is the output?

java

```java
int a = 5;
int b = a++ + a++;
System.out.println(a);   // ?
System.out.println(b);   // ?
```

```
a = 7   — incremented twice
b = 11  — first a++ uses 5 (a→6), second a++ uses 6 (a→7): 5+6=11
```

Q7. How to swap two numbers without a temp variable?

java

```java
// Using XOR
int x = 15, y = 27;
x = x ^ y;
y = x ^ y;
x = x ^ y;
System.out.println(x + " " + y);   // 27 15
```

```
// Using arithmetic
x = x + y;   // x = 42
y = x - y;   // y = 15
x = x - y;   // x = 27
// Risk: overflow for large numbers — XOR is safer
```

Q8. What is the output?

java

```
System.out.println(5 >> 1);    // ?
System.out.println(5 << 1);    // ?
System.out.println(-1 >>> 1);  // ?
```

```
2          — 5 / 2¹ = 2 (int division)
10         — 5 × 2¹ = 10
2147483647 — -1 is all 1s in binary; >>> fills with 0 → 0111...1111 =
Integer.MAX_VALUE
```

Q9. Can the ternary operator replace all `if-else`?

Technically yes for simple cases, but **no in practice**. Ternary is an **expression** (produces a value), while `if-else` is a **statement** (executes code). Ternary can't run multi-line logic, declare variables, or call void methods as a clean expression.

java

```
// ✅ Good use of ternary
String label = (score > 50) ? "Pass" : "Fail";

// ❌ Abuse — hard to read
String result = (a > b) ? (a > c) ? (a > d) ? "a" : "d" : "c" : "b";

// ✅ Prefer if-else for complex logic
if (a > b) {
    doThis();
    doThat();
} else {
    doOther();
}
```

Q10. What does `^` do — is it "power" in Java?

No — `^` is **XOR** (exclusive OR) in Java, not exponentiation. This is a common mistake:

java

```java
System.out.println(2 ^ 3);    // 1  — XOR of 2 and 3, NOT 2³=8!

// For power, use Math.pow()
System.out.println(Math.pow(2, 3));    // 8.0  ✅
```

Q11. What is the output?

java

```java
int x = 0;
boolean result = (x != 0) && (10 / x > 0);
System.out.println(result);   // ?
```

```
false — no exception!
&& short-circuits: (x != 0) is false, so right side never evaluated.
If & were used instead, ArithmeticException: / by zero.
```

Summary

```
Arithmetic   →  + - * / %          (watch int division & overflow)
Relational   →  == != > < >= <=    (use .equals() for objects)
Logical      →  && || !            (&& and || are short-circuit)
Bitwise      →  & | ^ ~            (operate on individual bits)
Shift        →  << >> >>>          (<< = ×2ⁿ, >> = ÷2ⁿ)
Ternary      →  ? :                (expression, not statement)

Key rules:
  ├── int/int = int  (always truncates)
  ├── postfix a++ returns old value, prefix ++a returns new
  ├── && short-circuits, & does not
  ├── ^ is XOR not power (use Math.pow for power)
  ├── >>> fills with 0, >> preserves sign bit
  └── final on reference ≠ immutable object
```